

Выпускные квалификационные работы бакалавра

Коновалов А. В.

9 сентября 2025 г.

Содержание

Темы курсовых работ	1
1. Эффективная реализация нормальных алгорифмов Маркова	1
2. Модель универсального транслятора, использующего СУО	2
3. Поддержка в Рефале-05 уплотнённого спискового представления	2
4. Базисное подмножество Рефала-5 с представлением данных, использующим раскрученные очереди	2
5. Классический суперкомпилятор ограниченного Рефала	3
6. Проектирование и реализация языка описания аппаратуры в стиле ASMD- FSMD	4

Темы курсовых работ

1. Эффективная реализация нормальных алгорифмов Маркова

Цель работы: написать, насколько это возможно, эффективную реализацию нормальных алгорифмов Маркова.

Наивная реализация — последовательный поиск подстрок в *строке* и выполнение замены неэффективен: поиск подстроки требует $O(n)$, замена в строке, представленной как *массив* символов, тоже требует $O(n)$, если длины искомой подстроки и её замены не совпадают.

Использование лексического распознавателя (автомата с размеченными заключительными состояниями) позволит осуществлять одновременный, а не поочерёдный поиск всех шаблонов.

Можно предложить двевовидную структуру данных, описывающую строку (листья — символы, внутренние узлы — конкатенации), и поиск, и замена в которой будут осуществляться со сложностью $O(h)$, где h — высота дерева. При использовании сбалансированного дерева сложность будет логарифмической.

Отдельным нетривиальным вопросом являются оптимизации алгорифмов — преобразования НАМ в эквивалентный, но вычисляющийся более эффективно (тут,

вероятно, не обойтись без помощи Антонины Николаевны). Другой нетривиальный вопрос — распараллеливание: определение условий, когда безопасно выполнять несколько замен за раз (и тут тоже).

Язык реализации — любой.

2. Модель универсального транслятора, использующего СУО

Цель работы: написать модельный универсальный транслятор, работающий по спецификации, описывающей произвольное синтаксически управляемое определение.

КС-грамматика на входе может быть произвольной — для построения её дерева вывода можно использовать алгоритм Эрли или GLR-парсер.

Для правил грамматики можно задавать произвольное количество семантических правил, вычисляющих произвольные атрибуты символов.

Синтаксис спецификации предстоит разработать. Приветствуется написание спецификации в форме EDSL для какого-либо языка программирования.

3. Поддержка в Рефале-05 уплотнённого спискового представления

Цель работы: сделать форк Рефала-05, использующий представление данных, описанное в этом сообщении:

<https://groups.google.com/g/refal/c/MRrNqwWDFNI>

Репозиторий исходного текста Рефала-05:

<https://github.com/Mazdaywik/Refal-05>

Допустимо сократить библиотеку встроенных функций — удалить функции, не используемые в тестовых примерах, предложенных преподавателем, в арифметических функциях не реализовывать (полу)длинную арифметику.

В данной работе придётся много писать на чистом Си и гораздо меньше, но тоже придётся, писать на Рефале.

4. Базисное подмножество Рефала-5 с представлением данных, использующим раскрученные очереди

Цель работы: сделать реализацию базисного подмножества Рефала-5, использующую раскрученные (bootstrapped) очереди для представления объектных выражений.

Допустимо сократить библиотеку встроенных функций — удалить функции, не используемые в тестовых примерах, предложенных преподавателем, в арифметических функциях не реализовывать (полу)длинную арифметику.

Раскрученная структура данных — обобщённый контейнер $C\langle T \rangle$, имеющий две реализации:

- мелкий контейнер, ограниченный по ёмкости $\text{Shallow}\langle T \rangle$,
- глубокий контейнер — контейнер мелких контейнеров $C\langle \text{Shallow}\langle T \rangle \rangle$.

Пример реализации на Java: <https://onlinegdb.com/7m9K1k2srz>.

В качестве представления объектных выражений предлагается использовать двустороннюю очередь, реализованную аналогичным образом:

- мелкая имеет фиксированную длину и организована как циклический буфер,
- глубокая представляет собой тройку из мелкой очереди, очереди мелких очередей (заполненные не менее чем наполовину) и ещё одной мелкой очереди.

Такая реализация обеспечивает амортизированные константные операции с концами и логарифмическую конкатенацию.

В качестве целевого языка рекомендуется использовать язык со сборкой мусора и обобщёнными типами, например, Java. Язык реализации — любой.

5. Классический суперкомпилятор ограниченного Рефала

Цель работы: написать суперкомпилятор ограниченного Рефала «по классическим рецептам» В. Ф. Турчина и А. П. Немытых.

Ограниченный Рефал — подмножество базисного Рефала, образцы которого сопоставляются за константное время, т.к. не содержат открытых е-переменных и повторных t- и е-переменных. Для этого подмножества несложно описать прогонку.

Суперкомпилятор должен реализовывать прогонку по классическим работам Турчина 1972 и 1974 года, для определения зацикливания достаточно использовать отношение Хигмана-Крускала (его можно дополнить и отношением Турчина, но не обязательно), для построения обобщения можно использовать алгоритм, описанный в книге Немытых (либо более продвинутый [алгоритмиз Рефала-5λ](#)).

Рекомендуемая литература:

- Анд.В. Климов, С.А. Романенко. Суперкомпиляция: основные принципы и базовые понятия // Препринты ИПМ им. М.В.Келдыша. 2018. № 111. С. 1–36. [doi:10.20948/prepr-2018-111](https://doi.org/10.20948/prepr-2018-111) PDF
- С.А. Романенко. Суперкомпиляция: гомеоморфное вложение, вызов по имени, частичные вычисления // Препринты ИПМ им. М.В.Келдыша. 2018. № 209. С. 1–32. [doi:10.20948/prepr-2018-209](https://doi.org/10.20948/prepr-2018-209) PDF
- В. Ф. Турчин. Эквивалентные преобразования рекурсивных функций, описанных на языке РЕФАЛ. В сб.: Труды симпозиума “Теория языков и методы построения систем программирования”, Киев-Алушта: 1972. Стр. 31-42. PDF
- В. Ф. Турчин. Эквивалентные преобразования программ на РЕФАЛе. В сб.: Труды ЦНИПИИАСС “Автоматизированная система управления строительством”, выпуск 6, М: 1974. Стр. 36-68. PDF
- V. F. Turchin. The School «Metacomputation in the Language Refal». Obninsk, July 11-23, 1990. The Basics of Metacomputation. (Chapter 3) PDF
- V. F. Turchin. The School «Metacomputation in the Language Refal». Obninsk, July 11-23, 1990. The Supercompiler. (Chapter 6) PDF

- А. П. Немытых. Суперкомпилятор SCP4: общая структура. Монография, М: Издательство URSS, 2007, ISBN 978-5-382-00365-8. — 152 с. [PDF](#)

На самом деле, всю эту рекомендуемую литературу читать не обязательно, обязательны первые три пункта, остальное рекомендуется пролистать.

6. Проектирование и реализация языка описания аппаратуры в стиле ASMD-FSMD

Не пугайтесь этой темы. Если относиться к Verilog'у как к языку программирования с необычной семантикой, то эта тема станет всего лишь курсовой по компиляторам. Собственно, она и есть курсовая по компиляторам в облачении незнакомой предметной области. Только комиссии это не говорите.

Цель работы: спроектировать язык с verilog-подобным синтаксисом, позволяющим описывать блок-схему ASMD в виде алгоритма на императивном языке, и реализовать для него компилятор в код на Verilog'е (содержащий, соответственно, сгенерированный FSMD).

Современные цифровые схемы проектируются при помощи языков описания оборудования (hardware describing language, HDL) — языков, напоминающих языки программирования для компьютеров, но компилирующихся в описание цифровой схемы (сильно упрощая: чертёж принципиальной электрической схемы, по факту это прошивки для ПЛИС, маски фотолитографии для ASIC'ов или голых чипов). Эти языки описывают схему на уровне регистровых передач (register transfer level, RTL) — примитивы языка описывают регистры, комбинационную логику (сумматоры, мультиплексоры, логические вентили, шифраторы, дешифраторы и т.д.) и связи между этими элементами. Это довольно низкий уровень с точки зрения описания алгоритмов (ниже него только уровень логических вентилях и уровень транзисторов).

Традиционная методика разработки схем подразумевает разделение устройства на устройство управления (часто описываемое при помощи конечного автомата: Мили или Мура) и тракт обработки данных (или операционное устройство, datapath). Разработчик должен построить конечный автомат, управляющий работой устройства, определить управляющие сигналы, которые автомат должен генерировать в состояниях (Мура) или переходах (Мили), и тракт обработки данных, в котором в определённых точках должны учитываться управляющие сигналы.

К этой PDF'ке приложены страницы из учебника Харриса и Харрис «Цифровая схемотехника и архитектура компьютера», демонстрирующие традиционную методику проектирования цифровых устройств.

Методика ASMD-FSMD предполагает проектирование устройства как конечного автомата с трактом данных (finite state machine with datapath, FSMD), для описания этого автомата составляется блок-схема (algorithmic state machine with datapath, ASMD).

Однако, в компьютерном программировании блок-схемы уже давно вытеснены императивными языками с развитыми управляющими конструкциями: если в языке программирования доступны только goto и if-goto, то без блок-схемы реализовать

сложный алгоритм сложно, если доступны if-then-else, while, for, break, continue и т.д., то блок-схема не нужна.

Предлагается разработать алгоритмический язык, который заменяет блок-схему ASMD, и компилятор для него в Verilog. *Иначе говоря, нужно сделать компилятор из языка с (почти) привычной императивной семантикой в язык с совсем непривычной (для программистов, не схемотехников) семантикой.*

Рекомендуемая литература:

- Соловьёв В. В. Основы языка проектирования цифровой аппаратуры Verilog. — 2-е изд., испр. и доп. — М.: Горячая линия — Телеком, 2024. — 284 с.: ил.

В этой книге нас особенно интересуют главы 22 и 23, однако в интернете мне удалось найти только PDF первого издания, в которой всего 18 глав. В главах 22 и 23 примерно то же самое, что и в статье ниже. Могу принести бумажную книгу.

- Соловьев, В. В. Использование методики ASMD-FSMD при проектировании на программируемых логических интегральных схемах устройств обработки сигналов / В. В. Соловьев // Радиотехника и электроника. – 2021. – Т. 66, № 12. – С. 1178-1188. – DOI 10.31857/S0033849421120184. – EDN FYXQRD. PDF, HTML
- Харрис Д. М., Харрис С. Л. Цифровая схемотехника и архитектура компьютера / Харрис Д. М., Харрис С. Л. ; Пер. с англ. Imagination Technologies. — 2-е изд., испр. — М. : ДМК, 2018. — 791 с. : ил. — Библиогр.: с. 769-770. — ISBN 978-5-97060-570-7.

<https://library.bmstu.ru/Catalog/Details/551376>

В интернете доступен PDF этой книги, адаптированный для чтения на мобильных устройствах, по-видимому, черновик. Скачать его можно отсюда:

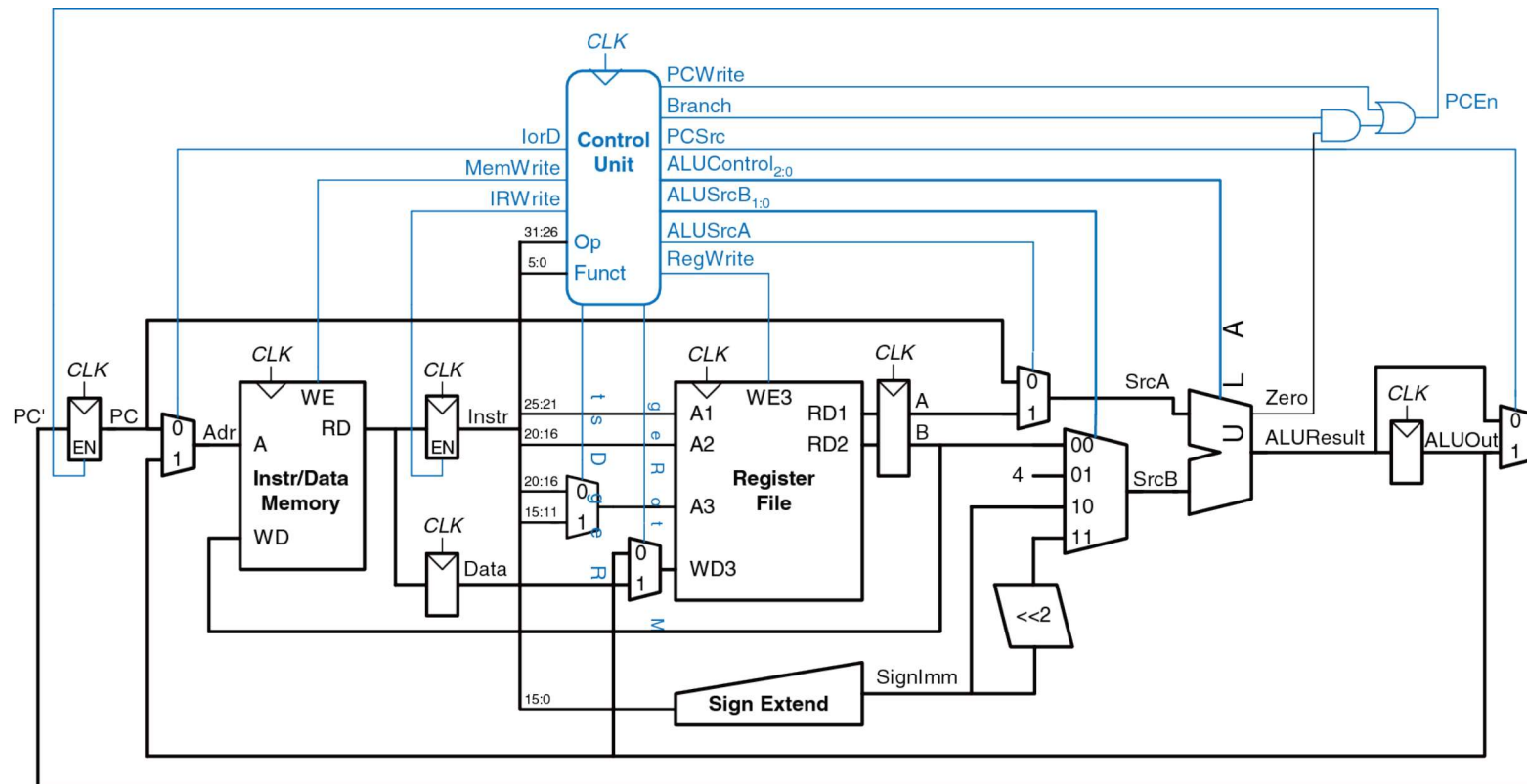
- Харрис, Д. М. Цифровая схемотехника и архитектура компьютера [Электронный ресурс] / Дэвид М. Харрис и Сара Л. Харрис. — Нью-Йорк : Elsevier. inc : Изд-во Morgan Kaufman, 2013. — on-line. — ISBN = 978-0-12-394424-5

<https://repo.ssau.ru/handle/Uchebnye-izdaniya/Cifrovaya-shemotekhnika-i-arhitektura-komputera-Elektronnyi-resurs-72425>

Из этой PDF'ки приложенные страницы.

Первые два пункта читать обязательно, третий обязательно пролистать.

Рис. 7.27 Законченный многотактный процессор MIPS



Как и в одноктактном процессоре, устройство управления поделено на две части, как показано на **Рис. 7.28**. Дешифратор АЛУ остается точно таким же, как и раньше; его таблица истинности приведена в **Табл. 7.2**.

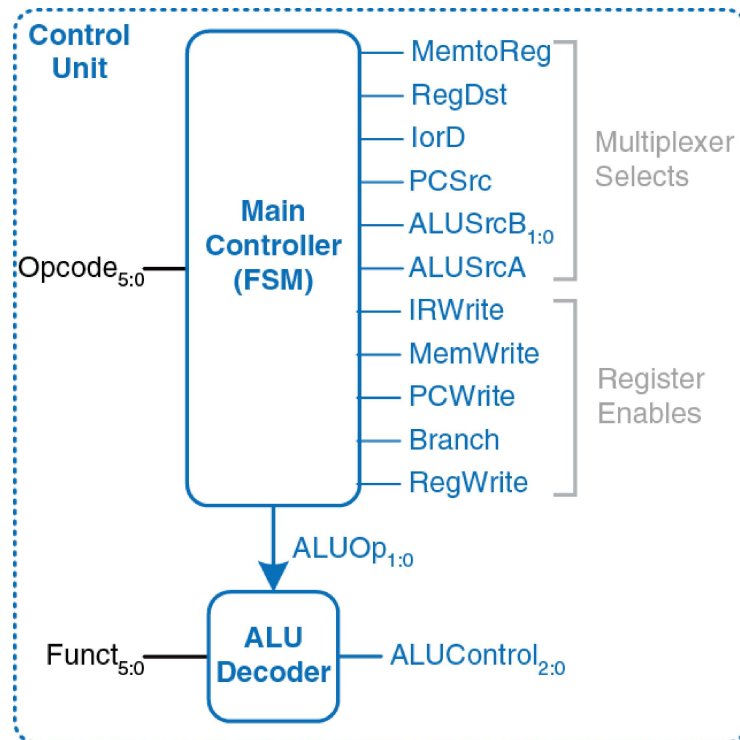
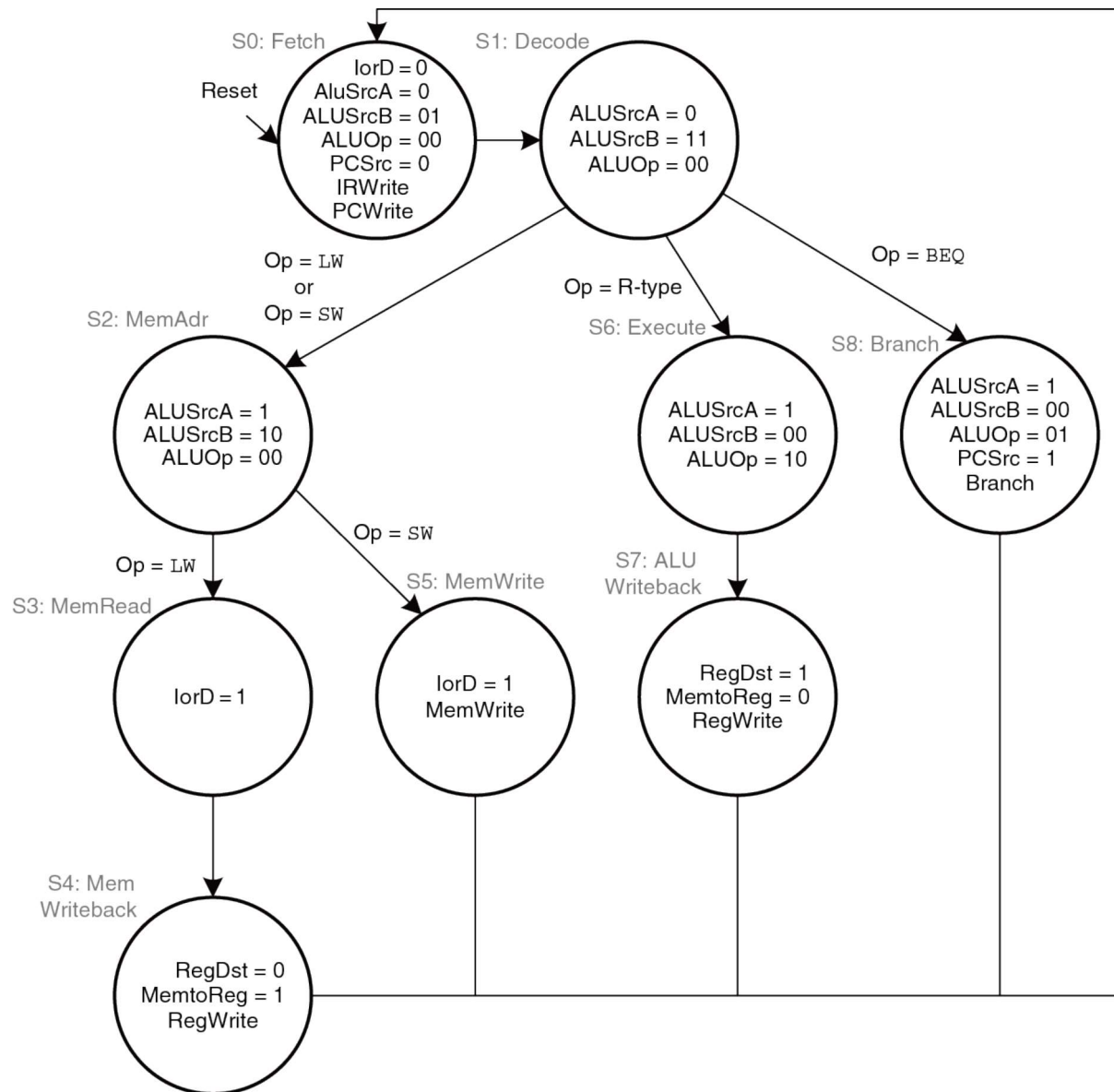


Рис. 7.28 Изменения в тракте данных для поддержки команд типа R

А вот вместо основного дешифратора нам понадобится конечный автомат, так как управляющие сигналы теперь будут зависеть не только от того, какая команда выполняется в данный момент, но и от того, на каком этапе выполнения она находится. Будем называть этот

**Рис. 7.39** Полная диаграмма состояний управляющего автомата